

Consider the following example:

Java

```
for(int i = 0; i<n; i++) {  
    for(int j = 0; j<n; j++) {  
        System.out.println("hello");  
    }  
}
```

Python

```
for i in range (0,n):  
    for j in range (0,n):  
        print("hello")
```

In the example above, the algorithm will repeat n times for the outer loop and n times for the inner loop, so a total of $n*n$ times, which means time complexity becomes $O(n^2)$.

Let's look at another example:

Java

```
for(int i = 0; i<n; i++) {  
    for(int j = 0; j<n+3; j++) {  
        System.out.println("hello");  
    }  
}
```

Python

```
for i in range (0,n):  
    for j in range (0,n+3):  
        print("hello")
```

In the example above, the number of repetitions is $n^2 + 3n$. As time complexity becomes more accurate for larger numbers, when n takes a very high value, the addition of $3n$ is not even taken into consideration. As such, constants are ignored (the rate of growth is what matters) and the lower-order terms are ignored (as n grows larger, the larger term dominates all other terms). Therefore, the Big O notation is still $O(n^2)$. In the same way, if the number of repetitions is $25n^3, n^3 + 25n^2 + \frac{1}{2}n$, the coefficients will be ignored and only the term with the highest exponent will be used, and so the Big O notation will be $O(n^3)$.

The final example is a simple statement, without any loops:

Java

```
System.out.println("enter a number: ");  
int value = reader.nextInt();  
sum = 50 + value;
```

Python

```
value = int(input("enter a number: "))  
sum = 50 + value
```

In this case, Big O notation is constant; no matter how large the input value, the algorithm will take the exact same time to run, so we can say the time complexity is $O(1)$.

The common time complexities expressed using Big O notation are:

- **$O(1)$ – constant time:** The algorithm performs a fixed number of operations; the time taken to run the algorithm does not depend on the size of the input. Such algorithms are ideal for operations where you need consistent performance regardless of input size, such as basic operations in data structures like hash tables.
- **$O(n)$ – linear time:** The algorithm scales linearly with the size of the input. Such algorithms are appropriate for simple searches where each element must be considered.

- **$O(n^2)$ – quadratic time:** The time taken to run the algorithm is directly proportional to the square of the input size. Such algorithms are often used when dealing with small data sets; they are inefficient for large data sets.
- **$O(2^n)$ – exponential time:** The algorithm's running time doubles with every increase in the input size. Such algorithms are generally impractical for large inputs due to rapid growth in execution time.
- **$O(\log n)$ – logarithmic time:** The algorithm's running time scales logarithmically with the increase in the input size. Such algorithms are suitable for searching and some divide-and-conquer algorithms. They are ideal for cases where you can efficiently reduce the problem size.

■ Space complexity

Space complexity analyses the amount of memory used by an algorithm with respect to its input size.

Common space complexities expressed using Big O notation are:

- **$O(1)$ – constant space:** The space used by the algorithm is not dependent on the size of the input. The algorithm uses a fixed amount of memory, no matter the input size.
- **$O(n)$ – linear space:** The memory usage scales linearly with the input size.
- **$O(n^2)$ – quadratic space:** The space taken to run the algorithm is directly proportional to the square of the input size.

Consider the following example:

Java

```
int sum = 0;
for(int i = 0; i < n; i++) {
    sum = sum + i;
}
System.out.println(sum);
```

Python

```
sum = 0
for i in range(0, n):
    sum = sum + i
print(sum)
```

In the example above, no matter how many repetitions there are, the space taken to store the `sum` and `i` variables in memory is always the same. The `sum` and `i` will be overwritten with every repetition, so the space complexity stays constant: $O(1)$.

As memory is not a real issue these days, it is often the case when writing an algorithm to aim for reducing the time complexity, even if that means trading off more space.

To calculate the space complexity of an algorithm, you need to look at:

- **Variables and constants:** As variables are overwritten and constants don't change their value during the execution of the program, they will always take up the same amount of space, so they don't need to be recalculated after the execution of the program.
- **Inputs:** Inputs are important for space complexity. If the inputs are variables, arrays or other data structures, their space complexity differs.
- **Execution:** Based on how the algorithm is written, the space complexity can be constant (when a fixed number of simple operations are performed) or differ (when, for example, a function calls itself several times, and so extra space is needed to store the return values and values of the parameters that will be used in unwinding to provide a solution).

Java

```
int sum = 0;
for(int i = 0; i < n; i++) {
    sum = sum + numbers[i];
}
System.out.println(sum);
```

Python

```
sum = 0
for i in range(0, n):
    sum = sum + numbers[i]
print(sum)
```

In the example above, the sum will include each value stored in the array, so the space required to run the algorithm is linear to the number of elements in the array: $O(n)$.

■ Choosing algorithms based on scalability and efficiency

- **Small data sets:** Simpler algorithms with higher time complexity (for example $O(n^2)$ or $O(n^3)$) can be acceptable.
- **Large data sets:** Algorithms with lower time complexity (for example $O(n \log n)$ or $O(\log n)$) are preferred, for better scalability.
- **Real-time requirements:** Algorithms with constant time complexity ($O(1)$) or logarithmic complexity ($O(\log n)$) are considered, where possible.
- **Complex problems:** Dynamic programming or divide-and-conquer approaches with manageable time complexity are used.

Selecting the right algorithm depends on the problem constraints, input size and performance requirements. Analysing time complexity helps ensure that the chosen algorithm will perform efficiently as the input size grows.

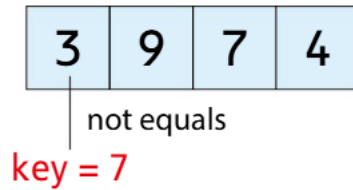
REVIEW QUESTIONS

- 1 Explain what Big O notation is and why it is used in Computer Science.
- 2 Define the terms “time complexity” and “space complexity”.
- 3 Outline the difference between $O(n)$ and $O(n^2)$. Identify two algorithms that present these complexities.
- 4 Explain why you think it is important to consider the worst-case space complexity of an algorithm.
- 5 Sketch a graph to compare the time complexity of $O(n)$ and $O(\log n)$. What differences do you notice?
- 6 Consider the following statement: “An algorithm with a Big O value of $O(n)$ will always be slower than one with a value of $O(\log n)$ ”. Evaluate this statement to true or false and explain your reasons.
- 7 Outline the two types of complexity analysis.
- 8 Identify which is generally faster for a very large n : $O(\log n)$ or $O(2^n)$.

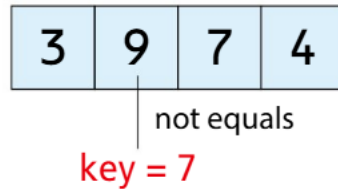
B2.4.2 Linear search and binary search

Search algorithms are used to find a specific item in a data structure. Such algorithms can be used to find an item in a list or in a database; to search for an item (word or phrase) in a document; to find a relevant webpage based on a keyword typed in a search engine; or to find a location on a map.

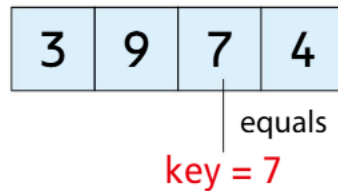
Step 1



Step 2



Step 3



■ 1D array (list) – linear search

◆ **Linear search:** a method of searching, in which each element is checked in sequential order.

Linear search

A **linear search** is also called a “sequential search” as it traverses a data structure from the beginning to the end when looking for a specific item. In the best-case scenario, the item is found in the first position, but in the worst-case scenario the item is the last item in the list and so the entire list is traversed.

Consider an array (list) of integers with ten elements and a key, being the element you are searching for.

To find the key, you will use a loop to traverse the array (list).

Starting at the first index, the element stored at position 0 will be compared with the key. If they are the same, the element is found; otherwise, the process repeats until the element is found or until the end of the array (list) is reached.

If a conditional loop is used, the algorithm can be stopped once the element is found. But if a count-controlled loop is used, the algorithm will traverse the entire list (array), even if the searched element has been found.

Java

```
int numbers= {3,9,7,4};
int key = 7;
int position = -1;
for(int i = 0; i<numbers.length; i++) {
    if(key==numbers[i]){
        position = i;
    }
}
if(position != -1){
    System.out.println("element found on position "+
position);
}
else{
    System.out.println("element was not found!");
}
```


Python

```
numbers = [3,9,7,4]
key = 7
position = -1
for i in range (0,len(numbers)):
    if key == numbers[i]:
        position = i
if position != -1:
    print("element found on position ", position)
else:
    print("element was not found")
```

This algorithm can be improved, so it stops when the element is found by using a conditional loop.

When you think of Big O notation, linear search has a time complexity of $O(n)$, as in the worst-case scenario every single element in the array (list) will be traversed in the attempt to find the searched key. At the same time, the space complexity is $O(1)$, as the space required to run the algorithm is constant.

Binary search

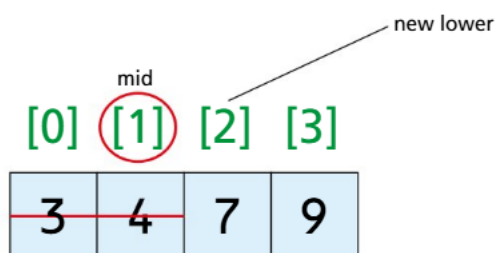
Linear search is not a very efficient algorithm, especially when the size of the array (list) is considerably high.

◆ **Binary search:** a method of searching an ordered array (list) by repeatedly checking the value of the middle element and disregarding the half of the data structure that does not contain the searched element.

Binary search is a more efficient searching algorithm as it reduces the number of searches to half with every comparison performed, so the time complexity is logarithmic $O(\log n)$. However, to be able to perform a binary search on an array (list), the data structure *must be sorted* (it must be in order). This is because the algorithm works as follows:

- Set a variable (**lower**) to store the lower bound index.
- Set a variable (**upper**) to store the upper bound index.
- Calculate the middle index (**mid**) by using the formula $\text{mid} = (\text{lower} + \text{upper}) / 2$
- Compare the value in the middle index with the search key (if `numbers[mid] == key`).
- If they are the same, the value is found and the algorithm can stop.

key = 7



```
lower = 0
upper = 3
mid = (lower + upper)/2 = 1
7 > 4, so
lower = mid + 1 = 2
mid = (lower + upper)/2 = 2
7 = 7, so the element is found
```

■ 1D array (list) – binary search

- If the value in the middle position is smaller than the key, you can disregard the left side of the array (if the array is sorted in ascending order) by setting the lower variable to the middle index + 1: $\text{lower} = \text{mid} + 1$
- If the value in the middle position is greater than the key, you can disregard the right side of the array (if the array is sorted in ascending order) by setting the upper variable to the middle index - 1: $\text{upper} = \text{mid} - 1$
- Repeat the entire process, starting from the third point until the value is found or the end of the array (list) is reached.

Java

```
int numbers[] = {3,4,5,7,9,11,13};
int key = 11;
boolean found = false;
int lower = 0;
int mid = 0;
int upper = numbers.length;
while((lower<=upper) && !found ) {
    mid = (lower+upper)/2;
    if (numbers[mid]==key) {
        found = true;
    } else if(numbers[mid]<key) {
        lower = mid+1;
    } else {
        upper = mid-1;
    }
}
if (found) {
    System.out.println("the value was found on position " + mid);
} else {
    System.out.println("the value was not found" );
}
```

Python

```
numbers= [3,4,5,7,9,11,13]
key =11
found = False
lower = 0
upper = len(numbers)
mid = 0
while lower<=upper and not found:
    mid = int((lower + upper)/2)
    if numbers[mid]==key:
        found = True
    elif numbers[mid]<key:
        lower = mid + 1
    else:
        upper = mid - 1
if found:
    print("the value was found on position ", mid)
else:
    print("the value was not found")
```

Each algorithm is more appropriate in different scenarios. For example, if the list (array) needs to be searched once for a given element, such as the ID of a worker, and the organization has fewer than 50 employees, it would be faster to simply use a linear search, rather than having to sort the data structure and then apply a binary search on it. However, if there is a need to search for the home address of a student based on their school ID, in a data structure already ordered based on the students' IDs, and the school has 1500 students, it would be much more appropriate to use a binary search to retrieve the student's details.

B2.4.3 Bubble sort and selection sort

Sorting refers to arranging the elements in an array (list) into ascending or descending order. You already practised a swapping technique at the beginning of the unit. This technique is used in sorting routines.

◆ **Bubble sort:** a sorting algorithm that compares adjacent values and swaps them if they are in an incorrect order.

■ Bubble sort

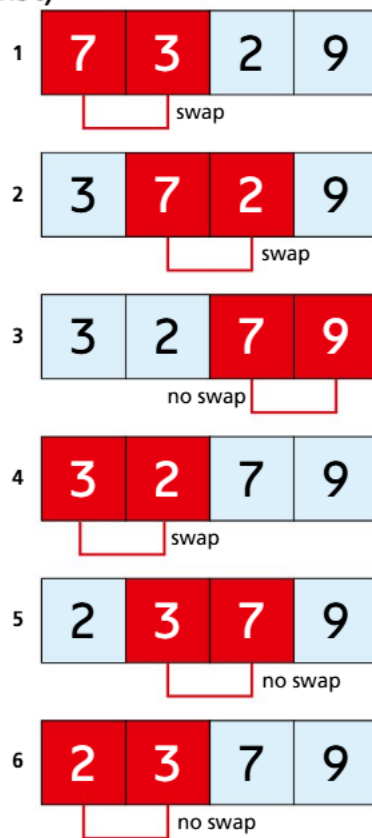
Bubble sort is a sorting algorithm that uses an incremental approach, and it works by repeatedly swapping the adjacent elements if they are not in the right order. If the array (list) was traversed only once, there might still be elements in the array (list) that are not yet sorted.

Bubble sort algorithms work as follows:

- Start at the beginning of the array (list).
- Compare the current element with the next one.
- If the two values are not in order, swap their contents.
- Move to the next element in the array (list).
- Repeat the process until all the elements have been sorted.

As shown in the diagram, once the array (list) has been traversed once, the last element is surely in the right position. Therefore, to optimize the algorithm, the number of repetitions can be reduced by 1.

Unsorted array (list)



Sorted array (list)



■ Bubble sort

Java

```
int numbers[] = {7,3,2,9};
for (int i = 0; i < numbers.length; i++) {
    for (int j = 0; j < numbers.length-1-i; j++) {
        if (numbers[j] > numbers[j+1]) {
            int temp = numbers[j];
            numbers[j] = numbers[j+1];
            numbers[j+1] = temp;
        }
    }
}
for (int i=0; i < numbers.length; i++) {
    System.out.print(numbers[i] + " ");
}
```

Python

```
numbers = [7,3,2,9]
for i in range(len(numbers)):
    for j in range(0, len(numbers)-1-i):
        if numbers[j] > numbers[j+1]:
            temp = numbers[j]
            numbers[j] = numbers[j+1]
            numbers[j+1] = temp
for i in range(len(numbers)):
    print(numbers[i], " ", end="")
```

The inner loop repeats from 0 to the size of the array (list) -1, because you compare one element with an adjacent one and, when the current element is the second last to be compared with the last element, that should be the last comparison that takes place.

The same repetition is reduced by *i* every time. This ensures that, when the array (list) is traversed the first time, it will repeat to its size -1, as *i* is initially 0, but with the next traversal it will repeat to size -2, and the next pass it will be size -3, and so on. This happens because, with the first pass of the array (list), the last element moves to the correct position; with the second pass, the last and the second last are in the correct position, and so on.

Another way to implement a bubble sort is by using a conditional loop:

Java

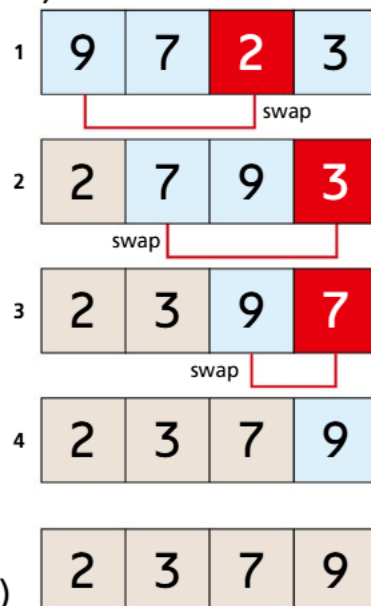
```
int numbers[] = {7,3,2,9};
boolean swapped = true;
int n = numbers.length;
while (n>0 && swapped) {
    swapped = false;
    n = n-1;
    for (int i = 0; i<n-1; i++) {
        if (numbers[i]>numbers[i+1]) {
            int temp = numbers[i];
            numbers[i] = numbers[i+1];
            numbers[i+1]=temp;
            swapped = true;
        }
    }
    for (int i=0;i<numbers.length; i++) {
        System.out.print(numbers[i]+" ");
    }
}
```

Python

```
numbers = [7,3,2,9]
swapped = True
n = len(numbers)
while(n>0 and swapped):
    swapped = False
    n=n-1
    for i in range(0, n-1):
        if numbers[i]>numbers[i+1]:
            temp = numbers[i]
            numbers[i]=numbers[i+1]
            numbers[i+1]=temp
            swapped = True
    for i in range(len(numbers)):
        print(numbers[i], " ", end=" ")
```

Although it is quite simple to understand the algorithm and to implement it, the bubble sort has a time complexity of $O(n^2)$, which means it is a very inefficient algorithm, especially when it comes to large sets of data. In terms of space complexity, the bubble sort algorithm is very efficient $O(1)$, requiring a constant memory space to store the variables read from the array, the indices and the temporary variable. This required space would not depend on the size of the input; it would not require additional space proportional to the size of the input array.

Unsorted array (list)



Sorted array (list)

■ Selection sort

Selection sort

To easily understand the algorithm, imagine you split the original array (list) into two parts. The first part is the sorted part, which is initially empty, and the second part is the unsorted part, which initially contains the entire array (list). With the first part, the smallest element in the unsorted part is selected and swapped with the first element in the array. This smallest element now becomes the sorted part of the array. In the second pass, you now search for the smallest element in the unsorted part and swap it with the second element. The sorted part now includes the first and second element. The process repeats until all the elements are sorted, the sorted part includes the entire array and the unsorted part is empty.

Java

```
int numbers[] = {9,7,2,3};
int min, minIndex;
for(int i = 0; i<numbers.length; i++) {
    min = numbers[i];
    minIndex = i;
    for(int j = i+1; j<numbers.length; j++) {
        if(numbers[j]<min) {
            min = numbers[j];
            minIndex=j;
        }
    }
    numbers[minIndex] = numbers[i];
    numbers[i] = min;
}
for(int i = 0; i<numbers.length; i++) {
    System.out.print(numbers[i] + " ");
}
```

Python

```
numbers = [9,7,2,3]
for i in range(len(numbers)):
    min = numbers[i]
    minIndex = i
    for j in range(i+1, len(numbers)):
        if numbers[j] < min:
            min = numbers[j]
            minIndex = j
    numbers[minIndex] = numbers[i]
    numbers[i] = min
for i in range(len(numbers)):
    print(numbers[i], " ", end="")
```

The same algorithm can be implemented by using just the index of the smallest element, instead of retrieving the smallest element and its index to swap:

Java

```
int numbers[] = {9,7,2,3};
int min;
for(int i = 0; i<numbers.length-1; i++) {
    min = i;
    for(int j = i+1; j<numbers.length; j++) {
        if(numbers[j]<numbers[min]) {
            min=j;
        }
    }
    int temp = numbers[i];
    numbers[i]=numbers[min];
    numbers[min]=temp;
    for(int i = 0; i<numbers.length; i++) {
        System.out.print(numbers[i] + " ");
    }
}
```

Python

```
numbers = [9,7,2,3]
for i in range(len(numbers)-1):
    min = i
    for j in range(i+1, len(numbers)):
        if numbers[j] < numbers[min]:
            min = j
    temp = numbers[i]
    numbers[i] = numbers[min]
    numbers[min] = temp
for i in range(len(numbers)):
    print(numbers[i], " ", end="")
```

◆ **Selection sort:** a sorting algorithm that repeatedly selects the smallest or largest element (ascending or descending order) from the unsorted part of the data structure and moves it to the sorted part.

The limitation of the **selection sort** is that it doesn't allow for an early exit if the array (list) is ordered at an earlier point. The time complexity of the selection sort is $O(n^2)$ and the space complexity is $O(1)$. Selection sort performs a smaller number of swaps; therefore, it is said to be a more efficient algorithm than the bubble sort algorithm. However, it is possible to stop the algorithm if all elements are sorted during an early pass in a bubble sort by using the flag in a conditional statement, which is not possible in a selection sort.

ACTIVITY

Research skills: Present information in a variety of formats and platforms – find an ingenious way to explain one of the programming algorithms you have studied. For example, use labelled cups to explain a sorting algorithm, create an animation, a video, and so on.

B2.4.4 Recursion (HL)

Recursion represents a technique that involves the use of functions, procedures or algorithms calling themselves one or more times until one or more specific conditions are met, at which point the process unwinds itself to produce a solution, by processing the last call to the first.

Characteristics of a recursive algorithm include:

- a method or function that calls itself
- a termination condition or a **base case** – a termination solution that is not recursive; without a base case, the algorithm will run to infinite
- a **general case** that calls itself recursively or is defined in terms of itself, and moves towards the base case by changing its state (**winding**)
- **unwinding**, which occurs when the algorithm reaches the base case (cascades up until the original problem is solved or, in other words, is processing the results, starting at the last call and building up towards the base case).

Recursive algorithms provide elegant solutions to complex problems, by often using less code and fewer variables than iterative approaches. They allow the programmer to divide complex problems into smaller sub-problems that are more readable and easier to solve. However, if many recursive calls are made, there is a heavy use of the stack, a process that is memory intensive and could potentially lead to stack overflow, and the computer running out of memory. If the termination condition is not set correctly, the algorithm might run to infinite, or the system might crash or freeze due to the high number of recursive calls.

Recursion might take longer to execute than other techniques or iterative approaches, as each call takes a specific amount of time, in addition to the time required to build up the final solution.

Recursion can also be challenging to follow sometimes, which can make it difficult for other programmers to maintain, document or modify it.

Recursion can be used:

- to implement sorting algorithms, such as quick sort
- for fractal image creation
- for traversing binary trees or graphs
- for solving mathematical problems, such as factorial functions and towers of Hanoi.

When choosing whether to solve a problem by using a recursive algorithm, ask yourself the following questions:

- Is it possible to identify a base case?
- Is it possible to solve the problem by calling itself or splitting it into smaller instances of the same problem?
- Does it require data structures like graphs, trees or linked lists – data structures that can be seen as repetitive instances of itself?
- Does it require backtracking?
- Is there a mathematical expression that can be translated into a recursive algorithm?
- Are you solving the problem in a more elegant, simpler and logical way by using recursion, without sacrificing too much memory or performance?

◆ **Base case:** a terminating solution (that is not recursive) to a process.

◆ **General case:** a process where the recursive call takes place.

◆ **Winding:** a process occurring when recursive calls are made until the base case is reached.

◆ **Unwinding:** a process occurring when the base case is reached, and the values are returned to build a solution.

REVIEW QUESTIONS

- 1 Define the term "recursion".
- 2 Compare recursive and iterative algorithms.
- 3 Explain the need for a base case in a recursive program.
- 4 Identify one consequence of not using a base case in a recursive function.
- 5 Identify some advantages and disadvantages of using recursion.
- 6 Explain what steps you would take to debug a recursive function that does not work as expected.

B2.4.5 Recursive algorithms (HL)

Factorial of a number

One of the exercises in Section B2.3.4 required you to find the factorial of a number. You were able to solve this problem by using a loop. However, now you understand the concept of recursion, you can easily establish that this problem can be solved by using a recursive algorithm. Starting from the mathematical formula that defines the factorial of n : $n! = n * (n-1)!$, you can identify that $5!$ can be defined as $5 * 4!$ and $4!$ can be defined as $4 * 3!$, and so on, until $n = 1$, which will return 1. The base case is when $n = 1$, as 1 will be returned, and the recursive call is when the method calls itself with $n - 1$ as a parameter. Therefore, the solution to this problem is:

Java

```
public static int factorial(int n) {
    if (n==1) {
        return 1;
    }
    else {
        return n * factorial(n-1);
    }
}

public static void main(String[] args) {
    int fact = factorial(5);
    System.out.println(fact);
}
```

Python

```
def factorial(n):
    if (n==1):
        return 1
    else:
        return n * factorial(n-1)
fact = factorial(5)
print(fact)
```

Handwritten notes for Python code:

- A bracket labeled "base" points to the `return 1` line.
- A bracket labeled "general" points to the `return n * factorial(n-1)` line.
- A red circle highlights the number 5 in `factorial(5)`.
- A recursive call tree is written in red: $5! \rightarrow 4! \rightarrow 3! \rightarrow 2! \rightarrow 1!$, with arrows indicating the sequence of calls.

Fibonacci sequence

Another application of a recursive method is in solving the Fibonacci sequence.

The Fibonacci sequence looks like this:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34...

THE FIBONACCI SEQUENCE

Each number is the sum of the two that precede it.

0 1 1 2 3 5 8 13 21

$$0 + 1 = 1$$

$$1 + 1 = 2$$

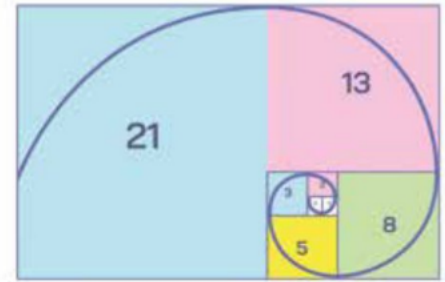
$$1 + 2 = 3$$

$$2 + 3 = 5$$

$$3 + 5 = 8$$

$$5 + 8 = 13$$

$$8 + 13 = 21$$



■ Fibonacci sequence

- $F_0 = 0$

- $F_1 = 1$

- $F_2 = F_1 + F_0$

- ...

- $F_n = F_{n-1} + F_{n-2}$

To find the sequence for the first n terms, the base cases are identified as being F_0 and F_1 , where the returned values would be 0 and 1 and the recursive call: $F_{n-1} + F_{n-2}$.

Java

```
public static int fib(int n) {
    if (n==0) {
        return 0;
    } else if (n==1) {
        return 1;
    } else {
        return fib(n-1) + fib(n-2);
    }
}

public static void main(String[] args) {
    Scanner read = new Scanner(System.in);
    System.out.println("Enter a number: ");
    int n = read.nextInt();
    for(int i = 0; i<n; i++) {
        System.out.print(fib(i) + " ");
    }
}
```

Python

```
def fib(n):
    if n==0:
        return 0
    elif n==1:
        return 1
    else:
        return fib(n-1) +
            fib(n-2)

n = int(input("enter a number: "))
for i in range(n):
    print(fib(i), " ", end="")
```


◆ **Quicksort:** a sorting algorithm that repeatedly selects an element as a pivot and partitions the other elements into two sub-arrays (lists): one that includes elements that are smaller than the pivot and the other one that includes elements that are larger than the pivot.

■ Quicksort

If the bubble sort and selection sort discussed earlier use an incremental approach, the **quicksort** is an efficient sorting routine that uses the divide-and-conquer algorithm. The divide-and-conquer principle refers to dividing the problem into two or more identical, smaller sub-problems that can be solved individually, and their solutions combined to produce the solution to the larger problem.

The quicksort algorithm makes use of a pivot element from the data set, against which the other elements are compared, to identify their correct position. The pivot element can be the first element, the last element, a random element or the middle element in the data set. Imagine that the data structure is broken into two partitions (sections): one that contains elements smaller than the pivot and one that contains elements larger than the pivot.

A way to construct the quicksort algorithm is by implementing the following logic:

- Set the pivot as the middle element.
- Start at each end of the list by using a left pointer and a right pointer.
- Move the values smaller than the pivot to the left partition.
- Move the values larger than the pivot to the right partition.
- Recursively apply the same principle for the left partition until all elements are sorted.
- Recursively apply the same principle for the right partition until all elements are sorted.

Java

```
public static void quickSort(int start, int finish, int[] numbers) {
    if(start >= finish) {
        return;
    }
    int left = start;
    int right = finish;
    int pivot = numbers[(start+finish)/2];
    System.out.print("start: " + start + " finish: " + finish + " left: " + left
    + " right: " + right + " pivot: " + pivot );
    while(left < right) {
        while(numbers[left] < pivot) {
            left = left+1;
            System.out.println("left: " + left);
        }
        while(numbers[right] > pivot) {
            right = right-1;
            System.out.println("right: " + right);
        }
        if(left <= right) {
            int temp = numbers[left];
            numbers[left] = numbers[right];
            numbers[right] = temp;
            left = left+1;
            right = right-1;
            System.out.println();
        }
    }
}
```

```

        for(int i = 0; i<numbers.length; i++) {
            System.out.print(numbers[i] + " ");
        }
        System.out.println();
        System.out.println("left: " + left);
        System.out.println("right: " + right);
    }
}
System.out.println("quickSort from start: " + start + " right: " + right);
quickSort(start, right, numbers);
System.out.println("quickSort from left: " + left + " finish: " + finish);
quickSort(left, finish, numbers);
}
public static void main(String[] args) {
    int numbers[] = {16,13,4,6,22,1,9,5};
    quickSort(0,numbers.length-1, numbers);
    for(int i = 0; i<numbers.length; i++) {
        System.out.print(numbers[i]+ " ");
    }
}

```

Python

```

def quickSort(start, finish, numbers):
    if(start>=finish):
        return
    left = start
    right = finish
    pivot = numbers[int>((start+finish)/2)]
    print("start: ", start, "finish: ", finish, "left: ", left, "right: ",
    right, "pivot: ", pivot)
    while left<right:
        while numbers[left]<pivot:
            left = left + 1
            print("left: ", left)
        while numbers[right]>pivot:
            right = right - 1
            print("right: ", right)
    if left<=right:
        temp = numbers[left]
        numbers[left] = numbers[right]
        numbers[right]= temp
        left = left + 1
        right = right - 1
    print()

```

```

for i in range (len(numbers)):
    print(numbers[i], " ", end="")
print()
print("left: ", left)
print("right: ", right)
print("quickSort from start: ", start, " to right: ", right)
quickSort(start, right, numbers)
print("quickSort from left: ", left, " to finish: ", finish)
quickSort(left, finish, numbers)
numbers = [16,13,4,6,22,1,9,5]
quickSort(0, len(numbers)-1, numbers)
for i in range(len(numbers)):
    print(numbers[i], " ", end="")

```

Common mistake

Always pay attention to the logic behind the algorithms you write. Many students implement sorting algorithms incorrectly or they confuse the algorithms in between. For example, when the question is requiring the sorting of an array in ascending order by using the selection sort, a common mistake is to implement an incorrect bubble sort, or an algorithm that outputs the values in descending order instead of ascending, as required.

Pay attention to the features of each of the sorting algorithms and understand the logic behind them.

1	[0] [1] [2] [3] [4] [5] [6] [7]	16 13 4 6 22 1 9 5	swap 16 and 5		start: 0 finish: 7 left: 0 right: 7 pivot 6
2	[0] [1] [2] [3] [4] [5] [6] [7]	5 13 4 6 22 1 9 16	swap 13 and 1		left: 1 right: 6 right: 5
3	[0] [1] [2] [3] [4] [5] [6] [7]	5 1 4 6 22 13 9 16	swap 6 and 6	quickSort start: 0 right: 2	left: 2 right: 4 left: 3 right: 3
4	[0] [1] [2] [3] [4] [5] [6] [7]	5 1 4 6 22 13 9 16	swap 5 and 1	quickSort start: 0 right: 0 quickSort left: 1 finish: 2	left: 4 right: 2 start: 0 finish: 2 left: 0 right: 2 pivot 1 right: 1
5	[0] [1] [2] [3] [4] [5] [6] [7]	1 5 4 6 22 13 9 16	swap 5 and 4	quickSort start: 1 right: 1 quickSort left: 2 finish: 2 quickSort left: 4 finish: 7	left: 1 right: 0 start: 1 finish: 2 left: 1 right: 2 pivot 5
6	[0] [1] [2] [3] [4] [5] [6] [7]	1 4 5 6 22 13 9 16	swap 22 and 9	quickSort start: 1 right: 1 quickSort left: 2 finish: 2 quickSort left: 4 finish: 7	left: 2 right: 1 start: 4 finish: 7 left: 4 right: 7 pivot 13 right: 6
7	[0] [1] [2] [3] [4] [5] [6] [7]	1 4 5 6 9 13 22 16	swap 22 and 16	quickSort start: 4 right: 3 quickSort left: 5 finish: 5 quickSort left: 5 finish: 7 quickSort start: 5 right: 6	left: 5 right: 3 start: 5 finish: 7 left: 5 right: 7 pivot 22 left: 6
8	[0] [1] [2] [3] [4] [5] [6] [7]	1 4 5 6 9 13 16 22		quickSort start: 5 right: 4 quickSort left: 6 finish: 6 quickSort left: 7 finish: 7	left: 7 right: 6 start: 5 finish: 6 left: 5 right: 6 pivot 13 right: 5 left: 6 right: 4

■ Quicksort

REVIEW QUESTIONS

- Complete a trace table for a bubble sort algorithm to sort the numbers: 16, 13, 4, 6, 22, 1, 9, 5.
- Complete a trace table for a selection sort algorithm to sort the numbers: 16, 13, 4, 6, 22, 1, 9, 5.
- Complete a trace table for a quicksort algorithm to sort the numbers: 16, 13, 4, 6, 22, 1, 9, 5.
- Create a table outlining the Big O value for both time and space complexities for all the searching and sorting algorithms you have studied.
- Outline the principles of working of a selection sort and a quicksort algorithm.

■ Traversal of binary trees

At this point, you will need to review the binary trees concepts in Section B4.1.4. If you have not covered this topic yet, take the time to do it now.

Binary trees can be traversed by using a recursive algorithm as well. The logic behind this is the following:

- Visit a node or display its content.
- Traverse the left subtree.
- Traverse the right subtree.

The order of those three operations depends on the type of traversal used. For example, in an in-order traversal you need to follow the left, root, right pattern, as shown in the algorithm below; as such you will go to the leftmost node and visit it, then the data in the node above, and then the right node. Once this is done, you move one level up and repeat the process, so the left subtree will be first traversed, then the root and then the right subtree. In a post-order traversal you need to follow the left, right, root pattern, and in a pre-order traversal you need to follow the root, left, right traversal.

Java

```
class Node
{
    int data;
    Node left;
    Node right;
    Node(int value){
        data = value;
        left=right=null;
    }
}
class testTraversal {
    void inorder(Node root)
    {
        if (root == null) {
            return;
        }
        inorder(root.left);
        System.out.println(root.data);
        inorder(root.right);
    }
    public static void main(String[] args) {
        Node root = new Node(5);
        root.left = new Node(3);
        root.right = new Node(8);
        root.left.left = new Node(2);
        root.left.right = new Node(4);
        root.right.left = new Node(6);
        root.right.right = new Node(9);
        inorder(root);
    }
}
```

Python

```
class Node:
    def __init__(self, value):
        self.data = value
        self.left = None
        self.right = None
    def inorder(root):
        if root is None:
            return
        inorder(root.left)
        print(root.data, " ", end="")
        inorder(root.right)
root = Node(5)
root.left = Node(3)
root.right = Node(8)
root.left.left = Node(2)
root.left.right = Node(4)
root.right.left = Node(6)
root.right.right = Node(9)
inorder(root)
```

PROGRAMMING EXERCISE

Implement the post-order and pre-order traversals of a binary tree using recursion.

ACTIVITY

Thinking skills:
Create novel solutions to problems – choose an iterative program you have created before, and attempt to rewrite it using a recursive approach.

ACTIVITY

Use your answer to the programming exercise above to answer the following questions.

- 1 Did you use two different functions to solve the exercise? Did the functions work as expected?
- 2 Did you use any local or global variables? Why did you need any local variables, if you used any?

**Common mistake**

Practical questions requiring algorithms to be described often lack clarity. Ensure steps are clearly outlined. You can support explanations by adding code or pseudocode statements, but those should be clearly explained to gain marks.

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ B2.5.1 Construct code to perform file-processing operations

When creating a program, you might want to store data permanently so that, once you close your program or switch off the computer and you turn it on again, the data can still be retrieved and manipulated. One way to achieve this is to store the data in a file. The program will be able to manipulate the data stored in several types of files, such as text files (.txt), comma-separated value files (.csv), binary files (.dat), and so on.

◆ **Absolute path:** the location of a file specified from the root directory (the full path).
 ◆ **Relative path:** the location of a file relative to the current folder.

To avoid having to use the **absolute path** of your file (to specify the entire location), save the file within the same folder as your program. This will allow you to access the file by simply using its file name, also called the **relative path** of the file.

A text file is a sequential file, meaning that data in the file can be accessed sequentially (line by line) and new records will be added to the end of the file.

A file can be opened in several modes: write, read and append. When opening a file in **write** mode, new data inserted into the file will overwrite the existing content in the file. **Read** mode is used to allow you to access the data from the file to read it, and **append** mode allows you to insert new records at the end of the file without erasing previous content.

The process to **write** text to a text file is as follows:

- Open the file for write or append, as needed.
- Prepare the line of text to be written.
- Write the line to the file.
- Close the file.

The process to **read** text from a text file is as follows:

- Open the file for read.
- Read the content from the file:
 - ☐ Read a single line from the file.
 - ☐ Use a loop to repeat for all the lines in the file and read them line by line.
 - ☐ Read the entire file into a data structure.
- Close the file.

File processing in Java

To process files in Java, you can use:

- the Scanner class
- the FileWriter class
- the BufferedReader class.

■ Scanner class

The Java package `java.io` includes the `Files` class that allows you to manipulate files, so this package needs to be imported first.

To create a file, follow the procedure of creating an object of type `File` and pass the file name as a parameter to the constructor.

```
File f = new File ("fileName.txt");
```

There are specific methods that can be used on the newly created file:

Method	Explanation
<code>exists()</code>	Returns true if the file exists on the disk
<code>delete()</code>	Deletes the file
<code>getName()</code>	Returns the name of the file
<code>length()</code>	Returns the number of characters in the file
<code>renameTo()</code>	Receives a parameter to specify the new name of the file
<code>canRead()</code>	Returns true if the file can be read

You have already used the `Scanner` class several times by now to read input from the keyboard. You can use the same class to read a file, by passing a file object as a parameter.

```
Scanner nameOfScanner = new Scanner (f);
```

In the line of code above, the `nameOfScanner` variable is an identifier, chosen to label the scanner for future use, and the `f` variable passed to the `Scanner` constructor is the file object created above.

The same purpose can be achieved by using:

```
Scanner nameOfScanner = new Scanner (new File ("fileName.txt"));
```

The `Scanner` uses tokens. When you use a function like `nextLine( )`, the `Scanner` will split the input into tokens (units of user input, separated by whitespaces).

For example, if the text file contains the following two lines of text:

```
"Bobby Bob"
```

```
3 75.5
```

the `Scanner` will split this into tokens, as follows:

Token	Possible data types
"Bobby	string
Bob"	string
3	int, double, string
75.5	double, string

Each call to the functions `next( )`, `nextInt( )`, `nextDouble( )`, and so on, will consume a token, meaning it will read the token and advance the cursor to the next one.

Consider the following exercise: Construct a program that will read five numbers from the text file called "numbers" and will output their average.

Consider that the text file includes the following numbers on five different lines:

3
3.4
4
4.6
5

Java

```
import java.util.*;
import java.io.*;

public static void main(String[] args) {
    try {
        Scanner read = new Scanner(new File("numbers.txt"));
        double sum = 0.0;
        double average = 0.0;
        for (int i = 0; i<5; i++) {
            double value = read.nextDouble();
            sum = sum + value;
        }
        average = sum/5;
        System.out.println("average is: " + average);
    } catch (FileNotFoundException e) {
        e.printStackTrace();
    }
}
```

In the code above, the function `nextDouble()` is used to read the next token in the file, as the problem already described the data type of the values stored in the file. But in case you don't know what type of data is stored, the scanner has functions that will return true or false when checking whether the next token is of a specific data type. As such, the function `hasNextInt()` will return true if there is a next token in the file of data type integer. Similarly, you can use `hasNextDouble()` or `hasNextLine()`.

Another structure used above is a FOR loop that repeats five times. This is possible as the number of lines in the text file is known. In case that is unknown information, a conditional loop can be used to repeat, as long as there is a next token in the file. The condition would include the function `hasNext()`.

To read a file line by line, the function `nextLine()` can be used.

Java

```
import java.util.*;
import java.io.*;

public static void main(String[] args) {
    try {
        Scanner read = new Scanner(new File("numbers.txt"));
        while(read.hasNext()) {
            String line = read.nextLine();
            System.out.println(line );
        }
    } catch (FileNotFoundException e) {
        e.printStackTrace();
    }
}
```

Even if the values stored in the file are of decimal numbers, they can be read with `nextLine()`, which interprets them as being of string data type. This is because, as described above, the values stored in a file can have more possible data types. If a conversion is possible to one of the required data types, there will be no error thrown.

In both examples, a `try/catch` block is used for exception handling. This is because the text file might not exist, or any other error might occur when reading the data and performing the required operations. By using exception handling techniques, the code will not crash.

The `java.io` package is used for both input and output. Therefore, it also includes the `PrintStream`, which allows you to write output to the text file.

Java

```
import java.util.*;
import java.io.*;

public static void main(String[] args) {
    try {
        PrintStream out = new PrintStream(new File
            ("numbers.txt"));
        out.println("Hello world!");
        out.println("I added content to the text file!");
    } catch (FileNotFoundException e) {
        e.printStackTrace();
    }
}
```

When creating the `PrintStream`, the `out` variable has been used as an identifier. The file passed as a parameter is generated by calling the constructor called `File`. In case this file already exists, it will be overwritten. Otherwise, a new file called `numbers.txt` will be created.

It is important to keep in mind that you should not open a file for both reading and writing at the same time (`Scanner` and `PrintStream`).

If you want to append data to the file so the new text will be added at the end of the file, rather than overwriting its contents, you can replace the `File` constructor with `FileOutputStream` and pass the parameter `true` together with the file name to it. By setting the second parameter to `true`, it sets the file to append mode.

Java

```
import java.util.*;
import java.io.*;

public static void main(String[] args) {
    try {
        PrintStream out = new PrintStream(new
            FileOutputStream("numbers.txt", true));
        out.println("A new line appended!");
        out.close();
    }
    catch (FileNotFoundException e) {
        e.printStackTrace();
    }
}
```

The `PrintStream` can be closed at the end, by simply using the `close()` function:

■ FileWriter class

The `FileWriter` class is also part of the `java.io` package, and it is used to write data in character form to the file (streams of characters). To create a `FileWriter`, simply create a `FileWriter` object that will pass the file name as a parameter to the constructor.

```
FileWriter out = new FileWriter("numbers.txt");
```

Again, if the file does not exist, a new one will be created, but if it exists it will be overwritten.

To write data to the file, the `write()` function is used and the writer can be closed by using the `close()` function.

Java

```
import java.util.*;
import java.io.*;

public static void main(String[] args) {
    try {
        FileWriter out = new FileWriter("numbers.txt");
        out.write("This is the first line!");
        out.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```


The code above will overwrite the existing text in the file. To open the file in append mode, just set the append mode to true, by adding a second parameter to the `FileWriter` constructor.

Java

```
import java.util.*;
import java.io.*;

public static void main(String[] args) {
    try {
        FileWriter out = new FileWriter("numbers.txt", true);
        out.write("This is the second line!");
        out.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

To read data from a file in character format, `FileReader` can be used.

```
FileReader read = new FileReader("numbers.txt");
```

This allows you to read a single character from the file with the `read()` method, to read the characters from the file and store them into a named array with the `read(char[] arrayName)`, or to read a given number of characters from the file starting at a specific index and store them into a named array with `read(char[] arrayName, int startIndex, int length)`.

Java

```
import java.util.*;
import java.io.*;

public static void main(String[] args) {
    char[] text = new char[100];
    try {
        FileReader reader = new FileReader("numbers.txt");
        reader.read(text);
        System.out.println(text);
        reader.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

■ BufferedReader class

`BufferedReader` class is used to read data from a character-based file. It can read a single character by using the `read()` function or read an entire line of text by using the `readLine()` function. To use the `BufferedReader`, there is a need to instantiate a `FileReader` object as well.

Java

```
import java.util.*;
import java.io.*;

public static void main(String[] args) {
    char[] text = new char[100];
    try {
        FileReader reader = new FileReader("numbers.txt");
        BufferedReader br = new BufferedReader(reader);
        int singleChar = br.read();
        while(singleChar!=-1) {
            System.out.print((char)singleChar + " ");
            singleChar = br.read();
        }
        br.close();
        reader.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

Before printing the character, it is converted to a char, as the values read from the `BufferedReader` are integer values representing the given character. If the method returns `-1`, it means there are no more characters in the file; therefore, this condition is included in the conditional loop.

Java

```
import java.util.*;
import java.io.*;

public static void main(String[] args) {
    char[] text = new char[100];
    try {
        FileReader reader = new FileReader("numbers.txt");
        BufferedReader br = new BufferedReader(reader);
        String line = br.readLine();
        while(line!=null) {
            System.out.println(line);
            line = br.readLine();
        }
        br.close();
        reader.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

File processing in Python

To process files in Python, use the following functions:

- `open( )`
- `read( )`
- `readline( )`
- `write( )`
- `close( )`

The `open( )` function takes two parameters: the name of the file and the mode the file should open in. There are several modes available in Python:

- `"w"` – write mode: The file is opened for writing text to it:
 - If the file does not exist, it creates a new file with the given name.
 - When adding text in write mode, the new text added will overwrite the previous text.
- `"a"` – append mode: The file is opened for adding text at the end of the file:
 - If the file does not exist, it creates a new file with the given name.
 - When adding text in append mode, the new text is added at the end of the file, so the previous text is not overwritten.
- `"r"` – read mode: The file is opened for reading data from it:
 - If the file does not exist, it will throw an error.

The default mode is the read mode. Therefore, writing `f = open("numbers.txt")` is the same as writing `f = open("numbers.txt", "r")`.

Python

```
f = open("numbers.txt", "w")
f.write("First line")
f.close()
```

The code above will create a file called `numbers.txt` in case it does not exist on the disk, and it will write the given text to it. Once this is done, the file is closed. If the program is run again with a different line of text, the existing text in the file will be overwritten.

Python

```
f = open("numbers.txt", "w")
f.write("First line")
f.close()
f = open("numbers.txt", "r")
print(f.read())
f.close()
```

The code above opens the file, writes the given text to the file and closes it. Afterwards, the file is open in read mode, the line is read and displayed, and then the file is closed.

`read( )` will read the entire content of the file so, if the file contains several lines, as shown below, they will all be displayed. Although this might seem an inefficient method, it can be

used to read the entire content of the file into a data structure, such as a list, and then to manipulate the data stored in the respective data structure.

Python

```
f = open("numbers.txt", "w")
f.write("First line\n")
f.write("Second line\n")
f.close()
f = open("numbers.txt", "r")
print(f.read())
f.close()
```

The `\n` will move the cursor on to the next line after a line of text has been written. To read a single line of text, the `readLine()` function can be used.

Python

```
f = open("numbers.txt", "w")
f.write("First line\n")
f.write("Second line\n")
f.close()
f = open("numbers.txt", "r")
print(f.readline())
f.close()
```

In the code above, although the file contains two lines of text, only the first one will be displayed. To display every line of text in the file, a conditional loop can be used.

Python

```
try:
    f = open("numbers.txt", "w")
    f.write("First line\n")
    f.write("Second line\n")
    f.close()
    f = open("numbers.txt", "r")
    text = f.readline()
    while text!="":
        print(text)
        text = f.readline()
except:
    print("There was a problem")
```

In the code above, a new line of text is read until the retrieved line is blank. As manipulating files might produce several errors, such as the file not being found on the disk, or an operation on the file being impossible to be completed, it is always a good idea to use exception handling via `try/except` blocks.

Python

```
try:
    f = open("numbers.txt", "w")
    f.write("First line\n")
    f.write("Second line\n")
    f.close()
    f = open("numbers.txt", "r")
    text = f.readline()
    while len(text) != 0:
        print(text)
        text = f.readline()
except:
    print("There was a problem")
```

The same result can be achieved by changing the condition of the loop to check whether the retrieved line of text has a length different from 0.

Another way to do this is to use the following loop:

Python

```
try:
    f = open("numbers.txt", "w")
    f.write("First line\n")
    f.write("Second line\n")
    f.close()
    f = open("numbers.txt", "r")
    for line in f:
        print(line)
except:
    print("There was a problem")
```

Top tip!

When creating programs that manipulate files, if you are using a relative path in your program, ensure that both the text file (.txt) and the coding file (.py or .exe) are within the same folder. If they are not within the same folder, the program won't be able to access that file unless the absolute path is provided.

This would repeat for each line in the file and display it accordingly.

Sometimes, you might want to check whether the file exists before attempting to perform an operation on it. This can be done by importing the `os` library and using the `os.path.exists` command:

Python

```
import os
if os.path.exists("numbers.txt"):
    print("the file exists")
else:
    print("the file does not exist")
```

It is very important to close the file once it has been manipulated for a given mode.

PROGRAMMING EXERCISES

- 1 Construct a program that checks whether a given file exists (called `name.txt`). If the file does not exist, the program asks the user what their name is, prints a personalized greeting message to them and saves the person's name into `name.txt`. If the file does exist, instead of prompting for their name, it loads the name from the file and prints the personalized greeting immediately.
- 2 Construct a program that uses a file to store a number to represent the number of times the program has run. Every time the program runs, it should increase the number by one and save the new value.
- 3 Construct programming code that generates 100 random numbers with values between 1 and 70. Store those numbers into a text file, one number per line. Call the file `numb.txt`.
- 4 Construct code that uses the `numb.txt` file to read the numbers, identify the number of duplicate values and delete those duplicate values from the file. Output how many numbers were deleted.
- 5 Construct code to append to the same file the exact number of elements that were deleted in question 4, including values from 71 to 150.
- 6 Construct programming code to read the text file and sort the values in ascending order using an efficient bubble sort algorithm.
- 7 Construct a program to read the values from the text file and identify the odd numbers. Store those numbers into a new text file.

● Linking questions

- 1 Does database programming in SQL require computational thinking? (A3)
- 2 Why is an understanding of variables and their scope important for effective memory management in computer systems? (A1)
- 3 Is algorithmic efficiency relevant to machine learning, where large data sets are processed, and computational cost can be significant? (A4)
- 4 Are data structures, such as stacks and queues, applicable in networking algorithms for packet routing and load balancing? (A2)
- 5 How can graph theory be applied to packet distribution in networks? (Mathematics A&I HL)
- 6 How do graph algorithms and terminologies, such as vertices and edges, impact machine learning algorithms like network analysis? (A4, Mathematics A&I HL)
- 7 How can network traffic be used as an example or connection to programming algorithms? (A2)
- 8 How can programming algorithms be used to develop machine learning methods? (A4)